

Reading Support: Connectedness

Companion to Week 2

Cem Bozsahin

LING487 · Fall 2026

Lambda notation exposes a dependency

Supports: A lambda term turns a dependency into an object

If M is an expression containing a placeholder x , then $\lambda x.M$ abstracts over that placeholder. The lambda marks what the expression awaits; the body shows what results after an admissible argument is supplied.

Lambda terms can themselves serve as arguments and results. This allows functions to be treated without requiring a permanent name for each one.

MOLR §1.4

Application substitutes without confusing binders

Supports: Beta reduction is disciplined substitution

Application places two lambda terms together. When the first has the form $\lambda x.M$, beta reduction replaces the free occurrences of x in M by the argument N : $(\lambda x.M)N =_{\beta} M[x/N]$.

Substitution must not capture a variable that was free in the argument. Bound variables can be renamed before reduction because their particular names are not structurally significant.

MOLR §1.4

Currying stages a multiple dependency

Supports: Currying replaces one tupled input with a sequence

A function $f : A \times B \rightarrow C$ accepts an ordered pair. Its curried counterpart accepts an A -value and returns a function awaiting a B -value: $f : A \rightarrow C^B$.

Nothing about the final values requires the arguments to arrive simultaneously. Currying makes their order of admission visible and provides the formal basis for treating partial combinations as functions.

MOLR §1.4

Raising allows an argument to participate functionally

Supports: Application can be factored through composition

Raising an object a gives $\lambda p.pa$, a function that awaits a function applicable to a . The combinator $\mathbf{T} = \lambda x.\lambda p.px$ performs this operation uniformly.

After raising, an argument can enter a compositional sequence. Application can consequently be obtained as the result of composition under the appropriate abstraction, rather than introduced as an unrelated primitive connection.

MOLR §1.4.5

Recursive and nonrecursive descriptions can be extensionally equal

Supports: Recursion is not the same thing as repetition

The recursive factorial definition refers to a smaller instance of the function: $n! = n(n-1)!$, with a base case at zero. The product $1 \times 2 \times \cdots \times n$ describes the same values without naming the function in its own definition.

Recursion therefore concerns the organization of a definition, not merely the presence of repeated material in its result.

MOLR §1.5

Finite expressions can instantiate an unbounded pattern

Supports: Linguistic embedding can be arbitrarily deep in description

Every uttered sentence has finite length. A grammar may nevertheless admit another clause in a position where the same kind of construction can occur again. No fixed finite bound follows from the structural description itself.

This claim must be kept separate from claims about memory, processing difficulty, or the length of sentences people ordinarily choose to utter.

MOLR §1.5

Probabilities presuppose possible outcomes

Supports: Probability requires a sample space before a number

A probability measure assigns values between zero and one to events, where events are subsets of a sample space Ω . The measure of Ω is one, and mutually exclusive events add.

The numerical result therefore depends on how the possible outcomes were delineated. Changing the corpus, population, or unit of observation can change both the events and their probabilities.

MOLR §1.6

Conditioning restricts the space under consideration

Supports: Conditional probability reverses only with evidence

For $P(B) > 0$, conditional probability is $P(A \mid B) = P(A \cap B) / P(B)$. We ask how much of the probability mass inside B also belongs to A .

Nothing in this definition makes $P(A \mid B)$ equal to $P(B \mid A)$. Confusing them reverses the direction of the condition and commonly exaggerates what evidence establishes.

MOLR §1.6

Logical validity preserves truth conditionally

Supports: Validity concerns the connection from premises to conclusion

An argument is valid when no interpretation makes all its premises true and its conclusion false. Modus ponens licenses B from A and $A \rightarrow B$. Modus tollens licenses $\neg A$ from $A \rightarrow B$ and $\neg B$.

Validity concerns the relation among propositions. Soundness additionally requires true premises.

MOLR §1.7

Quantification requires a domain and an interpretation

Supports: Quantifiers connect predicates with domains

In $\forall x[\text{child}(x) \rightarrow \text{slept}(x)]$, the variable ranges over a chosen domain. The predicate symbols must also receive interpretations before the formula can be evaluated as true or false.

Quantifier order can alter truth conditions. Formalizing a sentence therefore requires a claim about scope, not simply the replacement of words by symbols.

MOLR §1.7